Teoria de Grafos Trabalho 8

Enunciado

O trabalho consiste em implementar uma série de funções a serem colocadas no módulo Grafo7.hs, criado por você, cujos enunciados serão dados a seguir. Crie um arquivo de testes TesteGrafo7.hs que importe o módulo Grafo7.hs para testar cada uma dessas funções em pelo menos três grafos diferentes e que não sejam apenas variações do mesmo grafo. Se desejar, pode *importar* os módulos feitos por você nos trabalhos anteriores.

Em particular, use o módulo `GrafoValMatAdj.hs` criado por você no Trabalho 6, para resolver os exercícios deste trabalho.

Implemente as funções a seguir:

1. **representação b n** que recebe um número natural positivo `b`, representando uma base, e um número natural `n`, representando um natural decimal, e devolve uma lista com a representação `n` na base `b`. Para representar dígitos acima de 9, use a representação decimal deles. Assim, os dígitos da base 12 serão $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$. Alguns exemplos:

```
> representação 2 12 -- 12 na base binária
[1,1,0,0]

> representação 3 12 -- 12 na base 3
[1,1,0]

> representação 10 537 -- 537 na base 10
[5,3,7]

> representação 19 211 -- 211 na base 19
[11,2]

> representação 12 12345 -- 12345 na base 12
[7,1,8,9]
```

2. **completa xs n** que recebe uma lista `xs` de números inteiros e se `xs` possuir menos que `n` elementos, completa o início da lista com 0s. Caso contrário, devolve `xs` sem alterações. Exemplo:

```
> completa [1,1,1] 5
[0,0,1,1,1]

 > completa [1,1,1] 2
[1,1,1]
```

3. **conta b larg min max** que recebe uma natural positivo `b`, representando uma base e dois números naturais positivos: `larg`, `min` e `max`. A função devolve uma lista com todos os números entre `min` e `max`, inclusive, representados na base `b` e com largura dada por `larg`. Por exemplo

```
> conta 2 3 1 7 -- conta de 1 a 7 em binário usando números com 3 dígitos de largura
[[0,0,1], [0,1,0], [0,1,1], [1,0,0], [1,0,1], [1,1,0], [1,1,1]]
```

4. **coloraçõesKn n r** que retorna uma *lista* de grafos completos com `n` vértices, coloridos em arestas com todas as formas possíveis usando `r` cores. Represente as cores com números naturais entre 1 e r e os grafos usando matrizes de adjacências (valorados com as cores da arestas). Como exemplo, se a chamada a seguir for feita

```
> coloraçõesKn 3 2 -- todas as colorações de arestas do K3 usando 2 cores
```

os três primeiros grafos da lista poderiam ser

```
 [ -- todas as arestas na cor 1
GrAdj [1,2] (array ((1,1),(3,3))
[((1,1),0), ((1,2),1), ((1,3),1),
 ((2,1),1), ((2,2),0), ((2,3),1),
 ((3,1),1), ((3,2),1), ((3,3),0)]),

-- a aresta (2,3) está na cor 2 e as demais na 1
GrAdj [1,2] (array ((1,1),(3,3))
[((1,1),0), ((1,2),1), ((1,3),1),
 ((2,1),1), ((2,2),0), ((2,3),2),
 ((3,1),1), ((3,2),2), ((3,3),0)]),

-- a aresta (1,3) está na cor 2 e as demais na 1
GrAdj [1,2] (array ((1,1),(3,3))
[((1,1),0), ((1,2),1), ((1,3),2),
 ((2,1),1), ((2,2),0), ((2,3),1),
 ((3,1),2), ((3,2),1), ((3,3),0)]),

 ...
]
```

Dica: pense no contador do exercício anterior. Note que o número 0 é reservado para indicar a ausência da aresta.

5. **totalColoraçõesKn n r** que devolve um inteiro indicando o número total de colorações possíveis em um grafo completo com `n` vértices, coloridos em arestas com todas as formas possíveis usando `r` cores. Não use o exercício anterior, faças as contas que é mais rápido.

6. **combinações k xs** que recebe uma lista `xs` e um número natural k e devolva uma lista contendo todas as combinações possíveis dos elementos de `xs` combinados `k` a `k`. Por exemplo,

```
> combinações 2 [1,2,3]
[[1,2], [1,3], [2,3]]

> combinações 2 [1,2,3,4]
[[1,2], [1,3], [2,3], [1,4], [2,4], [3,4]]
```

7. **cliqueMonocromático g n** que recebe um grafo `g` com as arestas já coloridas e um número natural `n` e que devolve `Nothing` caso não encontre nenhum subgrafo K_n monocromático do grafo `g`, ou `Just ([v1,v2,..,vn], c)` caso encontre um K_n monocromático cujos vértices sejam $v1, v2, \cdots, vn$ com todas as arestas coloridas com a cor `c`.

8. **grafoRamsey s t n** que devolve uma bicoloração em arestas para o grafo K_n que não possui como subgrafos um K_s monocromático de uma cor ou um K_t monocromático de outra cor. A função deve devolver `Just g` onde `g` é um grafo colorido em arestas que satisfaz a condição anterior ou `Nothing`, caso não seja possível encontrar grafo algum que satisfaça a mesma condição. Para ver alguns grafos desse tipo, consulte `http://users.cecs.anu.edu.au/~bdm/data/ramsey.html`. Teste sua função para os valores abaixo:

s	t	n
5	1	3
10	2	3
3	3	5
3	4	8
3	4	9
5	5	6

Observação: existem algoritmos mais rápidos para resolver esse problema. Aqui usamos um método de força bruta para simplificar a implementação.